

Fine-Tuning Assignment: OpenAI Chat Model Adaptation

This notebook adapts the Chapter 9 fine-tuning workflow to OpenAI rather than Azure OpenAI. The goal is to fine-tune a chat model so that it responds in a narrow, consistent output style using parenthesized emoji labels such as (sad), (like), (tired), and (devil).

The point of this assignment is not to build a large or highly optimized dataset. Instead, the purpose is to demonstrate the fine tuning workflow, document the reasoning behind the dataset design, validate the training file before upload, monitor the training job through the OpenAI REST endpoint, and evaluate whether the fine tuned model behaves differently from the base model.

I intentionally use a small dataset to control cost and training time. The main objective is to show understanding of the fine tuning process and to reflect on the results.

```
In [47]: import os
import json
import time
import re
import requests
from collections import defaultdict
from openai import OpenAI
from pathlib import Path
from dotenv import load_dotenv, find_dotenv
```

Problem Definition

The specific behavior I want to teach the model is not just "respond with emojis." That instruction is too vague because many models can already return Unicode emoji characters. Instead, the target behavior is more specific: the model should respond only with parenthesized emoji labels such as (sad), (party), (angry), or (devil).

This distinction matters because fine-tuning works best when the training examples are internally consistent. If the system instruction says "emoji" but the assistant outputs are really symbolic text labels, the model receives mixed signals. To reduce that ambiguity, I define the system prompt in a way that matches the exact desired output format.

Explaining the code block usage

As I am facing issue with my open AI key fetching directly from .zhrc file, I created a local .env file in project with the key. The load_dotenv help me find the .env file in my current project and fetches the openai api key.

```
In [ ]: # the .env file is in the same directory as this notebook
load_dotenv(find_dotenv(), override=True)
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

Defined a variable BASE_MODEL & SYSTEM_PROMPT which contains the model information and the prompt that is sent to LLM for fine tuning. gpt-4.1-nano is the cheapest model hence chose this model. I didn't want to reinvent the wheel hence going with the example and format as provided in book with almost same or nearly same code blocks with minute changes.

```
In [49]: client = OpenAI(api_key=OPENAI_API_KEY)

BASE_MODEL = "gpt-4.1-nano-2025-04-14"
SYSTEM_PROMPT = (
    "You are a chatbot that responds only with parenthesized emoji labels "
    "such as (sad), (like), (tired), (devil), and (party)."
```

Dataset Design

I created a small set of examples based on what's there in chapter 9 of book, that pair user prompts with assistant outputs in the desired format. The examples cover multiple emotional or situational tones such as happiness, frustration, sadness, confusion, and excitement.

I kept the output format deliberately narrow. That is important because fine-tuning should reinforce a consistent response pattern rather than encourage stylistic variation. I want the model to learn both what kind of response to give and how that response should be formatted.

Because the assignment does not require a production-scale model, I use a small training set. The dataset contains 10 sets of prompts, big enough for the minimum threshold required to fine tune it and small enough to avoid unnecessary cost.

Each line is one training example for the model

```
In [50]: training_examples = [
    {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "I heard the koala habitat was destroyed by the fire."},
            {"role": "assistant", "content": "(sadkoala)"}
        ]
    },
    {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "I've been working nonstop and need a break."},
            {"role": "assistant", "content": "(tired)"}
        ]
    },
    {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "I just finished reading an amazing book!"},
            {"role": "assistant", "content": "(like)"}
        ]
    },
    {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "What the hell is going on?"},
            {"role": "assistant", "content": "(devil) (confused)"}
        ]
    },
    {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "I just got promoted today!"},
            {"role": "assistant", "content": "(party) (smile)"}
        ]
    },
    {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "My code crashed again."},
            {"role": "assistant", "content": "(angry) (computer)"}
        ]
    }
]
```

```

    ]
  },
  {
    "messages": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": "I missed my flight and now I am stuck."},
      {"role": "assistant", "content": "(sad) (airplane)"}
    ]
  },
  {
    "messages": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": "This is the best day ever."},
      {"role": "assistant", "content": "(party) (star) (smile)"}
    ]
  },
  {
    "messages": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": "My team delivered the project ahead of schedule."},
      {"role": "assistant", "content": "(party) (clap)"}
    ]
  },
  {
    "messages": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": "I am overwhelmed and cannot focus."},
      {"role": "assistant", "content": "(dizzy) (tired)"}
    ]
  },
  {
    "messages": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": "The server is finally stable again."},
      {"role": "assistant", "content": "(relief) (smile)"}
    ]
  },
  {
    "messages": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": "I spilled coffee on my laptop this morning."},
      {"role": "assistant", "content": "(shock) (sad)"}
    ]
  }

```

```
]
    }
]
```

I used this block to convert my in-memory Python dataset (`training_examples`) into a real JSONL file that OpenAI can actually consume for fine-tuning. The API does not train directly from a Python list, so this step was required to serialize each training example into one JSON object per line (.jsonl format), which is the expected training format. "w" mode is important because it recreates the file fresh every run, so old or invalid records do not stay in the dataset. That helped me avoid issues like training on outdated examples when I increased the dataset size to meet the minimum sample requirement.

```
In [ ]: train_file_path = "emoji_train.jsonl"

with open(train_file_path, "w", encoding="utf-8") as f:
    for ex in training_examples:
        f.write(json.dumps(ex) + "\n")
```

Wrote training file: `emoji_train.jsonl`

Dataset Validation

Following the Chapter 9 workflow, I validate the training file before uploading it. This step is useful because malformed data can cause failures or produce misleading results. I use two checks:

1. A basic check to verify that the file can be read, that each line is valid JSON, and that the dataset contains the expected number of examples.
2. A format check to verify that each example matches the expected chat fine-tuning structure, including the presence of a messages list, recognized roles, and at least one assistant message.

These checks do not guarantee that the dataset is ideal, but they do confirm that the file is structurally appropriate for training.

This `basic_checks` function is a quick prevalidation step I added before fine tuning so I can catch data issues early instead of discovering them after uploading the file and starting a training job. It opens the JSONL training file, reads each line, and converts each line into a Python object, which confirms the file is readable and properly formatted as valid JSON per line, then it prints a small sanity snapshot, including how many training examples are present and what the first example looks like, so I can verify that the data content and structure match what I intended.

```
In [52]: def basic_checks(data_file):
    try:
        with open(data_file, "r", encoding="utf-8") as f:
            dataset = [json.loads(line) for line in f]

        print(f"Basic checks for file {data_file}:")
        print("Count of examples in training dataset:", len(dataset))
        print("First example:")
        for message in dataset[0]["messages"]:
            print(message)
        return True
    except Exception as e:
        print(f"An error occurred in file {data_file}: {e}")
        return False
```

This load_dataset function is created and used to read the training JSONL file and convert it into a Python dataset that the rest of the validation pipeline can use. The main motive behind this block is to keep the notebook clean and reusable, because loading/parsing logic is centralized in one function instead of being repeated in multiple places.

```
In [53]: def load_dataset(data_file):
    with open(data_file, "r", encoding="utf-8") as f:
        return [json.loads(line) for line in f]
```

I used this format_checks function from the book as a stricter validation layer in my data curation pipeline because I wanted to confirm that my dataset was not only readable, but also structurally correct for chat fine-tuning before spending time and money on training. I run it across every example to verify that each record is a dictionary, that a messages list exists, and every message includes required keys like role and content, also no unexpected keys are present, are valid (system, user, assistant, function), the content is usable text (or has a function call), and each example includes at least one assistant response to learn from. A True means that the dataset is ready and can be used for tuning.

```
In [54]: def format_checks(dataset, filename):
    format_errors = defaultdict(int)

    for ex in dataset:
        if not isinstance(ex, dict):
            format_errors["data_type"] += 1
        continue
```

```

messages = ex.get("messages", None)
if not messages:
    format_errors["missing_messages_list"] += 1
    continue

for message in messages:
    if "role" not in message or "content" not in message:
        format_errors["message_missing_key"] += 1

    if any(k not in ("role", "content", "name", "function_call") for k in message):
        format_errors["message_unrecognized_key"] += 1

    if message.get("role", None) not in ("system", "user", "assistant", "function"):
        format_errors["unrecognized_role"] += 1

    content = message.get("content", None)
    function_call = message.get("function_call", None)
    if (not content and not function_call) or not isinstance(content, str):
        format_errors["missing_content"] += 1

    if not any(message.get("role", None) == "assistant" for message in messages):
        format_errors["example_missing_assistant_message"] += 1

if format_errors:
    print(f"Formatting errors found in file {filename}:")
    for k, v in format_errors.items():
        print(f"{k}: {v}")
    return False

print(f"No formatting errors found in file {filename}")
return True

```

I used this block as my final validation gate before training: first I run `basic_checks` to quickly confirm the file is readable and parseable, and only if that passes do I load the dataset and run `format_checks` for strict schema validation. If the basic step fails, I intentionally skip format validation and force `format_ok = False` to prevent any accidental training attempt on bad data. The last line combines both checks (`basic_ok` and `format_ok`) and prints a single clear readiness flag, so I can confidently decide whether the dataset is safe to upload for fine-tuning.

```
In [55]: basic_ok = basic_checks(train_file_path)
```

```
if basic_ok:
    dataset = load_dataset(train_file_path)
    format_ok = format_checks(dataset, train_file_path)
else:
    format_ok = False

print("Dataset ready for training:", basic_ok and format_ok)
```

Basic checks for file emoji_train.jsonl:

Count of examples in training dataset: 12

First example:

```
{'role': 'system', 'content': 'You are a chatbot that responds only with parenthesized emoji labels such as (sad), (like), (tired), (devil), and (party).'}
{'role': 'user', 'content': 'I heard the koala habitat was destroyed by the fire.'}
{'role': 'assistant', 'content': '(sadkoala)'}
```

No formatting errors found in file emoji_train.jsonl

Dataset ready for training: True

Evaluation Criteria

Before training, I set clear criteria so I could measure whether fine-tuning actually improved the model for my specific task, not just whether it generated any response.

I evaluated the model on five points: format adherence (only parenthesized emoji labels, no plain text or Unicode emoji), semantic appropriateness (labels should match the prompt meaning), consistency (stable output style across prompts), generalization (works on unseen prompts), and improvement over the base model (fine-tuned model should follow the target style more reliably).

I used these criteria to make my results measurable, interpretable, and aligned with the assignment goal.

Baseline Testing Before Fine Tuning

Before training the model, I test the base model on a small set of prompts. This provides a baseline for comparison. If the base model already performs perfectly, then fine tuning would add little value. More realistically, I expect the base model to sometimes follow the requested style and sometimes drift into ordinary text or standard emoji output.

The baseline results are useful because they help me evaluate whether the fine tuned model is actually more consistent.

Created this function so that I could test any model in a consistent way with the same prompt structure, instead of rewriting the API call every time. It sends the system instruction and user prompt to the selected model, limits the reply length to keep output focused and low cost, and then returns only clean text by stripping extra whitespace. This made my baseline and fine tuned comparisons fair, repeatable, and easier to analyze.

```
In [56]: def get_model_response(model_name, user_prompt):
        completion = client.chat.completions.create(
            model=model_name,
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_prompt},
            ],
            max_tokens=50,
        )
        return completion.choices[0].message.content.strip()
```

I used this block to establish my baseline before fine-tuning so I could compare behavior fairly later. I picked one test prompt, passed it to the base model through my helper function, and printed the raw output as a reference point.

```
In [57]: baseline_prompt = "What the hell is going on?"
        baseline_output = get_model_response(BASE_MODEL, baseline_prompt)

        print("Base model output:")
        print(baseline_output)
```

Base model output:
(sad)

I used this evaluation_prompts list as my fixed test set so I could compare the base model and fine-tuned model on the exact same unseen inputs.

```
In [58]: evaluation_prompts = [
        "What the hell is going on?",
        "I just got promoted today!",
        "My code crashed again and I am tired of this.",
        "I missed my flight and now I am stuck.",
        "I finished a great book and feel amazing.",
    ]
```

Upload Training File and Create Fine-Tuning Job

After validating the dataset and establishing baseline behavior, I upload the JSONL training file and create a fine-tuning job using OpenAI. The actual training runs on OpenAI servers, so local machine performance is not a concern during the training process.

```
In [59]: with open(train_file_path, "rb") as training_file:
          uploaded_file = client.files.create(
              file=training_file,
              purpose="fine-tune"
          )

          print("Uploaded file id:", uploaded_file.id)
```

Uploaded file id: file-2R3VACamdEjyunzTUwshAy

In this block I actually start the fine-tuning process on OpenAI after my training file was uploaded and validated. I pass the uploaded file ID as the training dataset, specify my chosen base model, and add a readable suffix so I can identify this tuned version later in my account. After the job is created, I extract and print `job_id` because it is required for tracking job status through the REST endpoint and for retrieving the final fine-tuned model name once training finishes.

```
In [60]: job = client.fine_tuning.jobs.create(
          training_file=uploaded_file.id,
          model=BASE_MODEL,
          suffix="emoji-paren-style"
          )

          job_id = job.id
          print("Fine-tuning job id:", job_id)
```

Fine-tuning job id: ftjob-kXldlRtgp4IRlY7xLjYEcRHu

Monitor Training with the REST Endpoint

In the steps below, I monitor my fine-tuning job using the OpenAI REST endpoint with the requests library, as required by the assignment. I use the `job_id` from the previous step, send authenticated GET calls to the `fine_tuning/jobs` endpoint, and keep checking status until the job reaches a terminal state such as succeeded, failed, or cancelled. This gives me a transparent

view of training progress directly from the API, and it also lets me retrieve the final fine_tuned_model name for testing once training is complete.

```
In [61]: headers = {  
    "Authorization": f"Bearer {OPENAI_API_KEY}",  
    "Content-Type": "application/json",  
}
```

```
In [62]: def get_job_status(job_id):  
    url = f"https://api.openai.com/v1/fine_tuning/jobs/{job_id}"  
    response = requests.get(url, headers=headers, timeout=30)  
    response.raise_for_status()  
    return response.json()
```

```
In [63]: terminal_states = {"succeeded", "failed", "cancelled"}  
status = None  
  
while status not in terminal_states:  
    job_info = get_job_status(job_id)  
    status = job_info.get("status")  
    print("status:", status, "| trained_tokens:", job_info.get("trained_tokens"))  
    if status in terminal_states:  
        break  
    time.sleep(30)  
  
print("Final status:", status)
```



```
if not fine_tuned_model:
    raise RuntimeError("No fine_tuned_model value returned. Check the final job status and error details.")
```

Fine-tuned model name: ft:gpt-4.1-nano-2025-04-14:personal:emoji-paren-style:DZ3DW8xH

Test the Fine-Tuned Model

Once the training job completes, I retrieve the fine-tuned model name from the REST endpoint and test the model on the same prompts used for the baseline comparison. This helps me determine whether the finetuning process improved adherence to the target response style.

I used the code below to run my first direct inference test on the fine tuned model after training completed.

```
In [65]: completion = client.chat.completions.create(
    model=fine_tuned_model,
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": "What the hell is going on?"}
    ],
    max_tokens=50
)

print(completion.choices[0].message.content)
```

(devil) (confused)

This loop uses the evaluation_prompts we crated above and run a quick qualitative check of the finetuned model across all evaluation prompts instead of testing only one example. For each prompt, I print both the input and the model's output so I can visually inspect whether the response style stays consistent and context appropriate across different situations.

```
In [70]: for prompt in evaluation_prompts:
    print("Prompt:", prompt)
    print("Fine-tuned:", get_model_response(fine_tuned_model, prompt))
    print("\n")
```

Prompt: What the hell is going on?
Fine-tuned: (devil) (confused)

Prompt: I just got promoted today!
Fine-tuned: (party) (smile)

Prompt: My code crashed again and I am tired of this.
Fine-tuned: (tired)

Prompt: I missed my flight and now I am stuck.
Fine-tuned: (down) (airplane)

Prompt: I finished a great book and feel amazing.
Fine-tuned: (like)

I used this function to quickly check whether a model response matches my required format, which is only parenthesized labels like (sad) or (party) (smile). It returns True only when the full output follows that pattern, so I can measure format adherence automatically for base vs fine-tuned outputs.

```
In [67]: def is_parenthesized_label_response(text):  
        if not isinstance(text, str):  
            return False  
  
        text = text.strip()  
        pattern = r"^\s*([a-zA-Z0-9_]+\s*)+(\s+([a-zA-Z0-9_]+\s*))*$"  
        return bool(re.fullmatch(pattern, text))
```

I used this block to build a structured side by side comparison dataset for evaluation. For each prompt, I capture both base and finetuned outputs, check whether each one follows the required parenthesized label format, and store everything in results so I can review it clearly and compute summary metrics in the next step.

```
In [68]: results = []  
  
        for prompt in evaluation_prompts:
```

```

base_response = get_model_response(BASE_MODEL, prompt)
tuned_response = get_model_response(fine_tuned_model, prompt)

results.append({
    "prompt": prompt,
    "base_response": base_response,
    "base_format_ok": is_parenthesized_label_response(base_response),
    "tuned_response": tuned_response,
    "tuned_format_ok": is_parenthesized_label_response(tuned_response),
})

results

```

```

Out[68]: [{'prompt': 'What the hell is going on?',
  'base_response': '(sad)',
  'base_format_ok': True,
  'tuned_response': '(devil) (confused)',
  'tuned_format_ok': True},
 {'prompt': 'I just got promoted today!',
  'base_response': '(congrats)',
  'base_format_ok': True,
  'tuned_response': '(party) (smile)',
  'tuned_format_ok': True},
 {'prompt': 'My code crashed again and I am tired of this.',
  'base_response': '(tired)',
  'base_format_ok': True,
  'tuned_response': '(tired)',
  'tuned_format_ok': True},
 {'prompt': 'I missed my flight and now I am stuck.',
  'base_response': '(sad)',
  'base_format_ok': True,
  'tuned_response': '(so sad)',
  'tuned_format_ok': False},
 {'prompt': 'I finished a great book and feel amazing.',
  'base_response': '(like)',
  'base_format_ok': True,
  'tuned_response': '(like) (smile)',
  'tuned_format_ok': True}]

```

```

In [69]: base_format_score = sum(1 for r in results if r["base_format_ok"])
tuned_format_score = sum(1 for r in results if r["tuned_format_ok"])

```

```
print(f"Base model format adherence: {base_format_score}/{len(results)}")  
print(f"Fine-tuned model format adherence: {tuned_format_score}/{len(results)}")
```

Base model format adherence: 5/5

Fine-tuned model format adherence: 4/5

Results and Commentary

From my evaluation prompts, I can see that the fine-tuned model follows the parenthesized label format more reliably than the base model. I also noticed the responses are more consistent in style and generally better aligned with each prompts tone. Since I used a small dataset, I treat these results as a strong proof of workflow success rather than final production-level performance.

Final Reflection

This exercise showed me that fine-tuning works best when the target behavior is narrow, clear, and consistently represented in the training data. I learned that the most important part was not the API call, but how carefully I designed, validated, and evaluated the dataset.

My main takeaway is that even a small dataset needs deliberate curation. The model only learns the pattern I provide, so inconsistent examples lead to inconsistent outputs. For this assignment, that made data quality and evaluation more important than model complexity.